Day 13 Student Notes: Claude for Ansible

> **Today in one line:** use Ansible to set up servers, let Claude help you write and review the playbook, and **prove it is safe by running it twice**.

Ansible in plain words

Ansible is a to-do list your servers follow. You write the wanted state once in YAML, and Ansible does it on one server or a hundred, over SSH, with nothing to install on the servers.

Word	Meaning
Inventory	The list of servers, in groups (web, db)
Playbook	A YAML file: which servers, and which steps
Task	One step, done by a module
Module	A ready-made tool: apt, copy, template, service
Role	A reusable folder of tasks, templates and variables
Handler	A step that runs only when something changed (for example restart nginx)
Variable	A value kept outside the steps (port, message)

Idempotent: run twice, nothing changes

A good playbook can run again and again safely. The second run changes nothing. Think: "make sure the light is on", not "press the switch".

Bad (shell)	Good (module)
shell: apt-get install -y nginx	apt: name=nginx state=present
shell: echo "Hi" > index.html	copy or template
command: service nginx restart	service module plus a handler

The proof: the PLAY RECAP shows **changed=0** on the second run.

A role is a tidy folder

```
roles/nginx/
  tasks/main.yml        the steps
  handlers/main.yml     restart nginx
  templates/            config files
  defaults/main.yml     default variable values
site.yml                runs the role on a group
```

```
# inventory.ini
[web]
localhost ansible_connection=local
```

Commands

```
ansible-playbook site.yml --syntax-check   is the YAML valid?
ansible-playbook site.yml --check          show what would change, change nothing
ansible-playbook site.yml                  do it
ansible-playbook site.yml -v               more detail when it fails
```

Always run twice. Use --check once the service is installed (on a brand new machine the service step cannot be checked yet).

Reading a failed run

Look at	What it tells you
TASK [name]	Which step failed
fatal: ... FAILED!	The error message
line and column	Where in your file
PLAY RECAP	ok, changed, failed counts per server

Ask Claude: what is wrong, which line proves it, what is the fix? Then **you** check the line.

Review checklist

Check	Why
Modules, not shell or command	Modules are idempotent
Second run shows changed=0	Proves it is idempotent
Every task has a name	Readable output
No passwords in the files	Use Ansible Vault or a secrets tool
Sensible permissions (never 777)	Anyone could change your files
Restart only through a handler	Restart only when the config changed

Set up

Ansible needs Linux, so we use a throwaway Ubuntu container (Docker Desktop, as on Day 11). Nothing on your laptop changes.

```
docker run -it --rm -v "$PWD":/work -w /work ubuntu:24.04 bash
apt-get update && apt-get install -y ansible curl
ansible --version
```

Type **exit** to leave; the container is deleted. Run playbooks only here, never on a real server. Windows PowerShell uses ${PWD} instead of $PWD.

The steps and prompts (also in Day13_Prompts_To_Paste.txt)

Step 1: a bad playbook

```
ansible-playbook bad-playbook.yml
ansible-playbook bad-playbook.yml
```

Look at the recap: it says changed both times. Then ask Claude:

```
Review bad-playbook.yml.
List every risky or poor practice, most serious first, with one line on why.
Then write a fixed copy as fixed-playbook.yml. Do not edit the original.
```

Step 2: Claude writes a role

```
Create an Ansible role called nginx in roles/nginx.
It should install nginx, write a site config and a home page from templates, and
make sure nginx is running.
Use modules, not shell. Put the port, web root and welcome message in
defaults/main.yml.
Also create site.yml that runs the role on the web group.
```

Check: apt module, template modules, service module, a handler, defaults/main.yml with the port, web root and message, and site.yml.

Step 3: run it twice, then change a variable

```
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml
ansible-playbook site.yml
curl localhost
ansible-playbook site.yml --check -e "nginx_message='Hello class'"
curl localhost
ansible-playbook site.yml -e "nginx_message='Hello class'"
curl localhost
```

You should see: first run changed; second run **changed=0**; the --check run says changed=1 but curl still shows the old page; the real run changes it and curl shows the new page.

Step 4: failed runs

```
A playbook run failed. Read the files in the case1_typo folder.
Tell me in simple words: 1) what is wrong, 2) the evidence line, 3) the fix.
Do not change any file yet.
```

```
Now fix playbook.yml in the case1_typo folder.
Change only what is needed and tell me what you changed.
```

```
Do the same for the case2_undefined folder.
What is wrong, what is the evidence line, and what is the fix?
```

Then do case3_yaml. Answers: **case1** package ngnix does not exist (fix: nginx). **Case 2** shop_nmae is undefined; the variable is shop_name. **Case 3** bad indentation at line 9: the second task is not lined up with the first.

Key words

Word	Meaning
Idempotent	Running it again changes nothing
Dry run (--check)	Shows what would change without changing it
Template	A file with variables, filled in by Ansible
become	Run the step with admin rights (sudo)
Ansible Vault	Encrypts secrets so they are not stored in plain text
Agentless	Nothing to install on the servers; Ansible uses SSH

Check yourself

1. What does idempotent mean?
2. Why is shell a poor choice for installing a package?
3. How do you prove a playbook is idempotent?
4. What does --check do?
5. A run failed. What three things do you look at?

Answers

1. Running it again leaves everything as it was: no changes.
2. It reports changed on every run and does not check first; the apt module does.
3. Run it twice. The second PLAY RECAP shows changed=0.
4. It shows what would change without changing anything.
5. The TASK name, the fatal message, and the line and column.

Take-home task (30 minutes)

1. Ask Claude to add a health page task to your nginx role (prompt below). Run the playbook twice and check that the second run shows changed=0.
2. Ask Claude how to use the role on two real servers (prompt below). Read the answer; do not run it on a real server.

```
Add a task to the nginx role that creates /var/www/html/health.html containing the
text OK.
Use a module, not shell. Keep it idempotent.
```

```
Explain in simple words how I would use this same role on two real servers.
Show an inventory.ini with a group called web. No passwords in any file.
```

Everything here is made up. Never put real passwords or server details into Claude or into a playbook.

> **Next class, Day 14:** Claude for Terraform: infrastructure as code.